

Ticket Booking

BookMyShow · Ticketmaster – ఈ series lo **అత్యంత చిన్న dataset**. 8 GB, ఒక laptop lo పడుతుంది. మరియు ఇదే అత్యంత కష్టమైన concurrency problem.

మూడు విరుపులు. మొదటిది – చదివి-ఆఫై-బుక్ చేస్తే 20,000 సీట్లకి **60,000 టికెట్లు** అమ్ముడవుతాయి. రెండోది – hold కి గడువు పెట్టకపోతే **60% సీట్లు ఖాళీగానే** ఉండిపోతాయి; గడువు చిన్నదైతే **78.8% చెల్లింపులు** డబ్బు తీసుకున్నాక విఫలమవుతాయి. మూడోది – retry తుఫాను **99.1% capacity** తింటుంది, మరియు అమ్మకం **ఒక్క tick కూడా** వేగంగా ముగియదు.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` `lo` run చేసినదే.

ఈ **PROBLEM** ఈ **SERIES** **LO** మిగతా అన్నిటికీ వ్యతిరేకం

ఇప్పటిదాకా ప్రతి doc `lo` సమస్య **పరిమాణం**: PB లు, కోట్ల requests, వందల machines.

ఇక్కడ మొత్తం data **8 GB**. సగటు load **సెకనుకి 12**. ఒక్క PostgreSQL instance దీన్ని నవ్వుతూ నడుపుతుంది.

మరియు ఇదే అత్యంత కష్టమైన problem – ఎందుకంటే ఆ 8 GB `lo` **ఒకే row** కోసం ఒకే క్షణంలో **20 మంది** పోటీ పడతారు, మరియు వాళ్ళలో **ఒక్కరే** గెలవాలి.

Scale ఒక సమస్య కాదు. Contention ఒక్కటే సమస్య.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఒకే సీటు, ఇద్దరు

3. Step – ఖాళీనా చూసి, ఆపై బుక్ చెయ్యి
4. **మొదటి విరుపు** – 20,000 సీట్లకి 60,000 టికెట్లు
5. Step – అణు ఆపరేషన్ · మరియు lock ఎంత పెద్దది కావాలి

Part 3 – రెండో విరుపు: పట్టుకుని వదలకపోవడం

6. Step – hold, ఆపై చెల్లింపు
7. **రెండో విరుపు** – సభ ఖాళీ, సీట్లు అన్నీ "అమ్ముడయ్యాయి"
8. Step – గడువు · చెల్లింపు తనిఖీ · మరియు గడువు ఎంత ఉంచాలి

Part 4 – మూడో విరుపు: అమ్మకం మొదలైన క్షణం

9. Step – లక్షల మంది, కొన్ని వేల సీట్లు
10. **మూడో విరుపు** – 99.1% పని వృథా
11. Step – వేచి ఉండే గది · మరియు ఒక ఆశ్చర్యకరమైన ఫలితం

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code · 19,750 holds · mutation testing
13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
14. నోటీస్ చెప్పాల్సిన English script

15. Follow-ups – పక్కపక్క సీట్లు, bots, డబ్బు వాపసు

16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

Shows, సీట్లు, వాడుకరులు. ఒక సీటు ఒక్కరికే అమ్మాలి. ఒక ప్రసిద్ధ show మొదలైన క్షణంలో లక్షల మంది ఒకేసారి వస్తారు.

2,00,000 shows • సగటున 400 సీట్లు

మొత్తం inventory rows : 80 మిలియన్ → ~8 GB

రోజుకి టికెట్లు : ~10,00,000 → సెకనుకి సగటున 12

ఇది ఈ series lo అత్యంత *చిన్న* dataset. ఒక్క laptop lo పడుతుంది.

సెకనుకి 12. ఒక MySQL instance సెకనుకి వేల writes చేస్తుంది. ఇక్కడ ఏ sharding, ఏ caching, ఏ replication చర్చా అవసరం లేదు.

ఇప్పుడు అదే system, అమ్మకం మొదలైన మొదటి సెకనులో:

కానీ ఒక ప్రసిద్ధ show అమ్మకం మొదలైన మొదటి సెకను:

వేచి ఉన్నవారు | సీట్లు | మొదటి సెకనులో requests | ఒక్కో సీటుకి పోటీదారులు

50,000	10,000		30,000		5
2,00,000	10,000		1,20,000		20
10,00,000	50,000		6,00,000		20

సగటు: సెకనుకి 12. శిఖరం: సెకనుకి 6,00,000. తేడా 51,840 రెట్లు.

మరియు ఆ శిఖరం మొత్తం *ఒకే show* మీద – ఒకే partition, అవే కొన్ని వేల rows.

సగటు నుంచి శిఖరానికి 51,840 రెట్లు - మరియు అది ఒకే చోట కేంద్రీకృతం

HLD 10 lo అర్థరాత్రి jobs ని ఎక్కువ shards తో పరిష్కరించలేమని చూశాం. ఇక్కడ అదే, ఇంకా తీవ్రంగా.

Sharding సహాయపడదు: ఆ show యొక్క నీట్లు ఒకే shard lo ఉండాల్సిందే - లేకపోతే "ఈ నీటు ఎవరిది" అనే నిర్ణయాన్ని machines మధ్య సమన్వయం చేయాలి.

Caching సహాయపడదు: cache పాతది, మరియు ఇక్కడ పాత జవాబు అంటే రెండుసార్లు అమ్మడం.

మీ సాధారణ HLD పనిముట్లు ఈ problem కి పనిచేయవు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఎక్కువ అమ్మడం (oversell) ఎప్పుడైనా ఫరవాలేదా?	విమానాలకి అవును, సినిమాకి కాదు. ఇది మొత్తం design ని మారుస్తుంది
వాడుకరి నీటు ఎంచుకుంటాడా, లేక "ఏదైనా 2 టికెట్లు"?	ఎంచుకుంటే contention ఒకే row మీద; లేకపోతే ఒక counter సరిపోతుంది
చెల్లింపు ఎంత సమయం తీసుకుంటుంది?	§8 – ఇదే hold గడువుని నిర్ణయిస్తుంది
పక్కపక్క నీట్లు కావాలా?	అవును అంటే ఇది ఒక నీటు కాదు, ఒక పరిధి – §15
అమ్మకం ఎప్పుడు మొదలవుతుందో ముందే తెలుసా?	తెలిస్తే §11 యొక్క వేచి ఉండే గది ముందే సిద్ధం చేయొచ్చు
ఒక వాడుకరి ఎన్ని టికెట్లు కొనొచ్చు?	పరిమితి లేకపోతే bots మొత్తం inventory తీసుకుంటాయి – §15
డబ్బు తీసుకున్నాక నీటు ఇవ్వలేకపోతే?	§8 lo కొలుస్తాం – మరియు ఆ రేటు సున్నా కాదు

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"Hold యొక్క గడువు చెల్లింపు సమయం కంటే తక్కువ అయితే ఏమవుతుంది?"

ఎందుకంటే §8 lo కొలుస్తాం: గడువు చిన్నదైతే 78.8% చెల్లింపులు డబ్బు తీసుకున్న *తర్వాత* విఫలమవుతాయి – నీటు అప్పటికే ఇంకొకరికి వెళ్ళిపోయింది.

మరియు గడువు పెద్దదైతే? అప్పుడు అమ్మకాలు తగ్గుతాయి – 9,820 నుంచి 6,355 కి. ఎందుకంటే చెల్లించనివాళ్ళ holds నీట్లని ఎక్కువసేపు బంధిస్తాయి.

ఈ ఒక్క సంఖ్యకి రెండు వైపులా నష్టం ఉంది, మరియు మధ్యలో ఒక optimum ఉంది.

Part 2 – మొదటి విరుపు: ఒకే సీటు, ఇద్దరు

3. Step – ఖాళీనా చూసి, ఆపై బుక్ చెయ్యి

ప్రతి booking system యొక్క మొదటి version:

```
const seat = await db.get('SELECT * FROM seats WHERE id = ?', seatId);
if (seat.owner !== null) return 'ఇప్పటికే అమ్ముడైంది';
// ... ఇక్కడ కొంత సమయం: template render, payment page, network ...
await db.run('UPDATE seats SET owner = ? WHERE id = ?', userId, seatId);
```

చదువు → తనిఖీ → రాయి. ఇది స్పష్టంగా సరైనది – ఒక్కరే వాడుతున్నంతవరకు.

ఆ మధ్యలో ఉన్న ఖాళీ – network, application logic, ఒక template render – అదే మొత్తం సమస్య.

4. మొదటి విరుపు – 20,000 సీట్లకి 60,000 టికెట్లు

1,000 సీట్లు, 3,000 మంది ఏకకాలంలో, "ఆలోచన" సమయం 3 ticks, 20 seeds:

విధానం	అమ్మిన సీట్లు	రెండుసార్లు అమ్మినవి	మొత్తం ticks
చదివి-ఆఫై-రాయడం	60,000	40,964	120

20,000 సీట్లు ఉన్నాయి • 60,000 టికెట్లు అమ్మాయి • 40,964 రెండుసార్లు

ప్రతి వాడుకరి సీటు ఖాళీగా ఉందని **సరిగ్గానే** చూశాడు. ఆ తర్వాత 3 ticks lo ఇంకో ఇద్దరు కూడా అదే చూశారు.

ముగ్గురూ "నాది" అని రాశారు. Database మూడు UPDATE లనూ **అనందంగా** అంగీకరించింది – ఎందుకంటే ప్రతి UPDATE విడిగా చెల్లుబాటు అయ్యేదే.

దోషం ఏ ఒక్క ఆపరేషన్ lo నూ లేదు. దోషం వాటి మధ్య ఖాళీలో ఉంది.

5. Step – అణు ఆపరేషన్ • మరియు lock ఎంత పెద్దది కావాలి

పరిష్కారం: తనిఖీ మరియు రాత **ఒకే అణు అడుగు** కావాలి. మూడు మార్గాలు, మరియు మూడూ పనిచేస్తాయి:

```
-- 1. పరతుత్ కూడిన UPDATE (optimistic - version తనిఖీ)
UPDATE seats SET owner = ?, version = version + 1
WHERE id = ? AND version = ? -- ← నేను చదివినప్పటి version
-- మారిన rows 0 అయితే: ఎవరో ముందు వచ్చారు

-- 2. Row lock (pessimistic)
SELECT * FROM seats WHERE id = ? FOR UPDATE

-- 3. మొత్తం show కి ఒక lock
SELECT * FROM shows WHERE id = ? FOR UPDATE
```

మూడూ **సరైనవి**. కానీ ఖర్చు ఒకటి కాదు:

విధానం	అమ్మిన నీట్లు	రెండుసార్లు అమ్మినవి	మొత్తం ticks
చదివి-అపై-రాయడం	60,000	40,964	120
version తనిఖీ (optimistic)	20,000	0	200
ఒక్కో నీటుకి lock	20,000	0	178
మొత్తం show కి ఒక lock	20,000	0	60,040

చివరి వరుస - సరైనది, మరియు 337 రెట్లు నెమ్మది

"మొత్తం show ని lock చేస్తే సులభం" అనేది ఒక సాధారణ interview జవాబు, మరియు అది **సరైన జవాబు** - correctness పరంగా.

కానీ అది అమ్మకాన్ని **పూర్తిగా వరుసలోకి** మార్చేస్తుంది: 178 ticks → **60,040 ticks**.

1,000 నీట్లకి ఇది ఒక అసాకర్యం. 50,000 నీట్లు stadium కి, ఒక్కో booking 200 ms తీసుకుంటే - **మొత్తం అమ్మకానికి 2.8 గంటలు**, మరియు ఆ సమయమంతా అందరూ వేచి ఉంటారు.

కాబట్టి నియమం: **LOCK** ని వీలైనంత ఇరుకుగా

ఒక్కో **నీటు** lock - వేర్వేరు నీట్లు కొనేవాళ్ళు ఒకరినొకరు ఆపరు. 1,000 నీట్లు show lo అది 1,000 స్వతంత్ర వరుసలు.

మరియు optimistic (version తనిఖీ) ఇంకా మెరుగు: అది **అసలు lock చేయదు**. ఢీకొంటే మళ్ళీ ప్రయత్నిస్తుంది. పోటీ తక్కువగా ఉన్నప్పుడు - అంటే **99.99% సమయం** - అది ఉచితం.

పై పట్టికలో optimistic (200 ticks) row lock (178) కంటే కొంచెం నెమ్మది, ఎందుకంటే ఢీకొన్నప్పుడు పని వృథా అవుతుంది. **అది శిఖర క్షణాల ఖర్చు, సాధారణ రోజుల ఆదా.**

Part 3 – రెండో విరుపు: పట్టుకుని వదలకపోవడం

6. Step – hold, ఆపై చెల్లింపు

ఒక సమస్య మిగిలింది: వాడుకరి సీటు ఎంచుకున్నాక చెల్లింపు పేజీకి వెళ్తాడు. అది 30 సెకన్ల నుంచి కొన్ని నిమిషాల వరకు.

ఆ సమయంలో ఆ సీటు ఎవరికీ ఇవ్వకూడదు. కాబట్టి రెండు దశలు:

1. hold(seat, user) → సీటు "పట్టుకున్నది" అవుతుంది
2. confirm(seat, user) → చెల్లింపు పూర్తయ్యాక "అమ్ముడైనది"

ఇప్పుడు ఒక ప్రశ్న, మరియు అది మామూలుగా అనిపిస్తుంది: చెల్లించకుండా వెళ్ళిపోయినవాళ్ళ సంగతి?

నిజ జీవితంలో అది మెజారిటీ. Cart abandonment రేట్లు 60-80%.

7. రెండో విరుపు – సభ ఖాళీ, సీట్లు అన్నీ "అమ్ముడయ్యాయి"

500 సీట్లు × 20 seeds = 10,000 సీట్లు, 5,000 మంది (సీట్ల కంటే 10 రెట్లు డిమాండ్), 60% మంది hold తీసుకుని వెళ్ళిపోతారు:

ఎర్పాటు	అమ్మినవి	అమ్ముడుపోని ఖాళీ సీట్లు	రెండుసార్లు అమ్మినవి
hold కి గడువు లేదు	3,986	6,014 (60%)	
0			

డిమాండ్ 10 రెట్లు ఉంది • 60% సీట్లు ఖాళీగానే ఉండిపోయాయి

ఒక్క సీటు కూడా రెండుసార్లు అమ్మలేదు – ₹5 పనిచేసింది. మరియు system ఇప్పటికీ **ఘోరంగా విఫలమైంది**.

కారణం: hold తీసుకుని వెళ్ళిపోయినవాడు ఆ సీటుని **శాశ్వతంగా** బంధించాడు. System కి అది "పట్టుకున్నది" – అమ్మలేదు, ఖాళీ చేయలేదు.

Show రోజున సభలో **60% కుర్చీలు ఖాళీగా** ఉంటాయి, మరియు website "అన్నీ అయిపోయాయి" అని చూపిస్తుంది.

ఇది "ఏదీ తప్పు జరగలేదు" నియమాలన్నిటినీ నెగ్గుతుంది. Oversell లేదు, lost update లేదు, corruption లేదు. కేవలం **ఏమీ జరగలేదు**.

8. Step – గడువు • చెల్లింపు తనిఖీ • మరియు గడువు ఎంత ఉంచాలి

స్పష్టమైన పరిష్కారం: **hold కి ఒక గడువు (TTL)**. కానీ అది ఒక కొత్త race ని తెరుస్తుంది:

వాడుకరి A సీటు పట్టుకున్నాడు. గడువు ముగిసింది. B ఆ సీటు తీసుకున్నాడు. ఇప్పుడు **A యొక్క చెల్లింపు పూర్తయింది** – డబ్బు తీసుకున్నాం.

వివరాలు	అమ్మినవి	అమ్ముడుపోని ఖాళీ సీట్లు	రెండుసార్లు అమ్మినవి
hold కి గడువు లేదు	3,986	6,014 (60%)	
0			
గడువు 200 • చెల్లింపు తనిఖీ లేదు	12,064	4 (0%)	
2,068			
గడువు 200 • చెల్లింపు తనిఖీ ఉంది	9,371	629 (6%)	
0			
గడువు 400 • చెల్లింపు తనిఖీ ఉంది	9,820	180 (2%)	
0			

రెండో వరుస – గడువు పెట్టాం, మరియు ₹4 యొక్క విరుపు తిరిగి వచ్చింది

10,000 సీట్లకి **12,064 టికెట్లు**, అందులో **2,068 రెండుసార్లు**.

ఎందుకంటే చెల్లింపు పూర్తయినప్పుడు code ఇలా అంది: "చెల్లింపు విజయవంతం → సీటు ఇవ్వు". అది **hold ఇంకా నాదేనా** అని చూడలేదు.

₹5 lo hold ని అణువుగా చేశాం. కానీ confirm ని చేయలేదు.

సరైన **confirm** నాలుగు విషయాలు తనిఖీ చేయాలి, మరియు నాలుగూ **ఒకే అణు అడుగులో**:

```
confirm(seat, user, now) {  
  if (this.#state[seat] === SOLD) return this.#holder[seat] === user;    // idempotent
```



```

    if (this.#state[seat] !== HELD) return false;
    if (this.#holder[seat] !== user) return false;
    if (this.#until[seat] <= now) return false;
    ముగిసిందా?
    this.#state[seat] = SOLD;
    return true;
  }

```

// ఇది నాదేనా?
// గడువు

ఆ మొదటి పంక్తి – **idempotency** – తప్పనిసరి, ఎందుకంటే చెల్లింపు gateway యొక్క webhook రెండుసార్లు రావచ్చు. (HLD Deep 09 lo ఇదే కొలిచాను.)

ఇప్పుడు అసలు ప్రశ్న: **గడువు ఎంత ఉంచాలి?**

hold గడువు	అమ్మినవి	అమ్ముడుపోని సీట్లు	డబ్బు తీసుకుని సీటు ఇవ్వలేనివి
100	7,248	2,752	26,988 (78.8%)
200	9,371	629	10,169 (52.0%)
400	9,820	180	0 (0.0%)
600	9,506	494	0 (0.0%)
1200	7,798	2,202	0 (0.0%)
3000	6,355	3,645	0 (0.0%)

రెండు వైపులా నష్టం • మధ్యలో ఒక OPTIMUM

గడువు చాలా చిన్నది (100) → చెల్లింపు పూర్తయ్యేలోపే hold ముగుస్తుంది. **78.8% చెల్లింపులు** డబ్బు తీసుకున్నాక విఫలం. ప్రతి ఒక్కటీ ఒక refund, ఒక support ticket, ఒక కోపంగా ఉన్న వాడుకరి.

గడువు చాలా పెద్దది (3000) → చెల్లించనివాళ్ళు holds సీట్లని ఎక్కువసేపు బంధిస్తాయి. అమ్మకాలు 9,820 నుంచి **6,355 కి – 35% తక్కువ**.

ఉత్తమం 400 – 9,820 అమ్మకాలు, 0 విఫల చెల్లింపులు. మరియు 400 అనేది **గరిష్ట చెల్లింపు సమయం** (ఈ నమూనాలో 400).

కాబట్టి గడువు ఒక ఏకపక్ష సంఖ్య కాదు – దానికి ఒక నియమం ఉంది

hold గడువు ≥ చెల్లింపు యొక్క p99 సమయం.

దాని కంటే తక్కువ అయితే మీరు **మీ నెమ్మదైన వాడుకరులకే** విఫలమవుతారు – అంటే నెమ్మదైన network, పాత ఫోన్, బ్యాంక్ OTP కోసం వేచి ఉన్నవాళ్ళు.

దాని కంటే ఎక్కువ అయితే మీరు **inventory ని వృథా** చేస్తున్నారు, మరియు ఆ ఖర్చు సరళంగా పెరుగుతుంది.

కాబట్టి మీరు కొలవాల్సినది **"గడువు ఎంత"** కాదు – **"చెల్లింపు p99 ఎంత"**. రెండేది తెలిస్తే మొదటిది వస్తుంది.

Part 4 – మూడో విరుపు: అమ్మకం మొదలైన క్షణం

9. Step – లక్షల మంది, కొన్ని వేల సీట్లు

ఇప్పుడు system సరైనది: oversell లేదు, inventory కరగదు. మిగిలించి శిఖర క్షణం.

2 లక్షల మంది, 10,000 సీట్లు. Server సెకనుకి 2,000 requests నిర్వహించగలదు. 60% మంది hold తీసుకుని చెల్లించరు, కాబట్టి సీట్లు తిరిగి వస్తూ ఉంటాయి – అమ్మకం కొన్ని వందల ticks నడుస్తుంది.

సహజమైన client ప్రవర్తన: విఫలమైతే వెంటనే మళ్ళీ ప్రయత్నించడం.

10. మూడో విరుపు – 99.1% పని వృథా

2,00,000 మంది • 10,000 సీట్లు • server సెకనుకి 2,000 requests
60% మంది hold తీసుకుని చెల్లించరు • hold గడువు 120 ticks • 8 seeds

విధానం	మొత్తం requests	వృథా %	అమ్మడయ్యే వరకు ticks
విఫలమైతే వెంటనే మళ్ళీ	29,36,250	99.1%	1,469
ఘాతీయ backoff	19,26,701	98.7%	1,472
వేచి ఉండే గది (waiting room)	1,96,568	87.3%	1,471

29 లక్షల REQUESTS • అందులో 99.1% వృథా

అమ్మల్సినవి 10,000 సీట్లు. 60% మంది చెల్లించరు కాబట్టి ~25,000 holds కావాలి.

System **29,36,250 requests** నిర్వహించింది – అవసరమైన దానికి **117 రెట్లు**.

మరియు ఆ వృథా కేవలం డబ్బు కాదు – అది **capacity**. ఆ 29 లక్షల requests మీ servers, మీ database, మీ connection pool అన్నిటినీ నింపుతాయి, మరియు నిజంగా కొనగలిగే వాడుకరి **వాటి వెనుక వరుసలో** నిలబడతాడు.

11. Step – వేచి ఉండే గది • మరియు ఒక ఆశ్చర్యకరమైన ఫలితం

మొదటి స్పష్టమైన పరిష్కారం **ఘాతీయ backoff** – విఫలమైన కొద్దీ ఎక్కువసేపు ఆగడం. అది requests ని 29 లక్షల నుంచి **19 లక్షలకి** తెచ్చింది – 34% ఆదా, మరియు వృథా ఇప్పటికీ **98.7%**.

నిజమైన పరిష్కారం **వేచి ఉండే గది**: వాడుకరులని booking system లోపలికి రానివ్వకపోవడం. మిగిలిన సీట్లకి తగినంత మందిని మాత్రమే లోపలికి వదలడం; మిగతావాళ్ళు ఒక వరుసలో ఒక స్థానం చూస్తారు.

1,96,568 requests – retry తో పోలిస్తే **15 రెట్లు తక్కువ**.

కానీ చివరి నిలుపు వరుస చూడండి – మూడూ ఒకే సమయం

1,469 · 1,472 · 1,471 ticks. **వ్యత్యాసం 0.2%**.

అంటే ఆ 29 లక్షల requests అమ్మకాన్ని **ఒక్క tick కూడా వేగంగా** చేయలేదు.

ఎందుకంటే అమ్మకం వేగాన్ని నిర్ణయించేది requests సంఖ్య కాదు – **hold గడువు**. సీట్లు 120 ticks కి ఒకసారి మాత్రమే తిరిగి వస్తాయి. ఎంతమంది ఎన్నిసార్లు అడిగినా ఆ రేటు మారదు.

Retry తుఫాను 15 రెట్ల infrastructure ఖర్చు పెట్టి, సున్నా లాభం ఇస్తుంది.

ఇది ఒక సాధారణ నమూనా, మరియు దీన్ని గుర్తించడం విలువైనది

ఒక system యొక్క throughput ని ఒక **బాహ్య** అంశం నిర్ణయిస్తున్నప్పుడు (ఇక్కడ hold గడువు; HLD 11 lo మర్యాద ఆలస్యం; HLD 13 lo partition సంఖ్య) –

ఎక్కువ requests పంపడం ఆ పరిమితిని మార్చదు. అది కేవలం మీ ఖర్చుని మారుస్తుంది.

కాబట్టి సరైన ప్రశ్న "ఎక్కువ requests ఎలా నిర్వహించాలి" కాదు – **"ఆ requests అసలు ఎందుకు వస్తున్నాయి"**.

వేచి ఉండే గది వాటిని **నిర్వహించదు** – **అది వాటిని పుట్టకుండా ఆపుతుంది**, మరియు వాడుకరికి "మీరు వరుసలో 40,312వ వారు" అని ఒక **నిజమైన** జవాబు ఇస్తుంది.

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code • 19,750 holds • mutation testing

ఎనిమిది నియమాలు: ఒక సీటు ఇద్దరికీ అమ్మకూడదు, గడువు ముగియని hold ని లాక్ చేయకూడదు, గడువు ముగిశాక confirm కాకూడదు, confirm idempotent, ఖాళీ+పట్టుకున్నవి+అమ్ముడైనవి = మొత్తం, చివరికి holds మిగలకూడదు, అమ్ముడైన సీటుకి సరైన యజమాని, expire() చెప్పినన్ని సీట్లు నిజంగా ఖాళీ కావాలి.

500 యాదృచ్ఛిక runs • 19,750 holds • 12,124 అమ్మకాలు
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

holds	19,750
confirms	12,124
expiries	5,130
reconfirms	6,036
steals	0
releases	2,409
rejected	6,65,464
stale	78

ఆ rejected 6,65,464 ముఖ్యమైనది – 6.6 లక్షల ప్రయత్నాలు సరిగ్గా తిరస్కరించబడ్డాయి. అంటే పోటీ నిజంగా జరిగింది, మరియు రక్షణలు నిజంగా పనిచేశాయి.

==== mutation testing ====

మార్పు లేని code	→ 0/500 విఫలం
confirm lo యజమాని తనిఖీ తీసేస్తే	→ 31/200 విఫలం
ఉదా: సీటు 19: u13 కి అమ్మకం u14 కి కూడా	
confirm lo గడువు తనిఖీ తీసేస్తే	→ 120/200 విఫలం
ఉదా: సీటు 12: గడువు 620, ఇప్పుడు 623 – అయినా u33 కి ఇచ్చాం	
hold lo version తనిఖీ తీసేస్తే (చదివి-ఆపై-రాయడం)	→ 0/200 విఫలం
hold lo "ఇప్పటికే అమ్ముడైంది" తనిఖీ తీసేస్తే	→ 200/200 విఫలం
ఉదా: సీటు 22: అమ్ముడైన సీటుని u41 పట్టుకున్నాడు	
hold lo గడువు లేని hold ని కూడా లాక్కుంటే	→ 157/200 విఫలం
ఉదా: సీటు 29: u47 యొక్క గడువు ముగియని hold ని u16 లాక్కున్నాడు	
expire చేసినప్పుడు సీటుని తిరిగి ఖాళీ చేయకపోతే	→ 156/200 విఫలం
ఉదా: expire 1 అని చెప్పింది, holds 1 → 1	
confirm ని idempotent కాకుండా చేస్తే	→ 199/200 విఫలం
ఉదా: సీటు 7: u7 కి అమ్మకం రెండో confirm విఫలమైంది	

ఒక మార్పు బతికింది - **VERSION** తనిఖీ - మరియు కారణం ఆలోచించదగినది

hold() నుంచి version తనిఖీని తీసేస్తే **0/200** విఫలం. అది 78 సార్లు పనిచేసింది (stale: 78), కానీ ఏ నియమాన్నీ కాపాడలేదు.

కారణం: నా Show class యొక్క పద్ధతులు **అణువులు** - hold() లోపల చదవడం మరియు రాయడం మధ్య ఎవరూ ప్రవేశించలేరు. కాబట్టి state తనిఖీలే (SOLD ? HELD మరియు గడువు ఉందా?) సరిపోతాయి.

అంటే version నంబరు అనవసరమా? కాదు - నేను దాన్ని అవసరం లేకుండా చేశాను.

నిజమైన systems lo చదవడం మరియు రాయడం **రెండు వేర్వేరు network calls**. ఆ ఖాళీయే §4. Version తనిఖీ అనేది ఆ ఖాళీ ఉన్నప్పుడు అణుత్వాన్ని **తిరిగి తెచ్చే** పద్ధతి.

కాబట్టి దీనికి సాక్ష్యం ఈ fuzz lo లేదు - అది **§4 యొక్క 40,964** lo ఉంది. నా model ఆ సమస్యని ఊహించి తొలగించేసింది.

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-6	Requirements · scale సమస్య కాదని చూపించడం	8 GB · సెకనుకి 12 · శిఖరం 51,840 రెట్లు
6-12	చదివి-ఆపై-రాయడం → oversell	20,000 సీట్లకి 60,000 టికెట్లు
12-18	అణు ఆపరేషన్ · lock ఎంత ఇరుకుగా	show lock సరైనది, 337 రెట్లు నెమ్మది
18-26	hold → గడువు లేకపోతే 60% ఖాళీ	డిమాండ్ 10 రెట్లు, సీట్లు 60% అమ్ముడుపోలేదు
26-33	confirm తనిఖీ · గడువు ఎంత	తనిఖీ లేకపోతే 2,068 రెండుసార్లు · optimum 400
33-40	Retry తుఫాను	99.1% వృథా · 29 లక్షల requests
40-45	వేచి ఉండే గది · మరియు అది వేగం పెంచదు	15 రెట్లు తక్కువ requests, అదే 1,470 ticks

ఈ **PROBLEM LO** మిమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "hold కి TTL పెడతాను" అని చెప్పి ముందుకి వెళ్తారు. అది సరైనది, మరియు అది **ఒక సంఖ్యని ఎంచుకోకుండా** చెప్పడం.

మీరు ఆగి ఇలా చెప్పండి: "TTL కి రెండు వైపులా నష్టం ఉంది. నేను కొలిచినప్పుడు - TTL చాలా చిన్నదైతే **78.8% చెల్లింపులు** డబ్బు తీసుకున్నాక విఫలమయ్యాయి, ఎందుకంటే hold చెల్లింపు కంటే ముందే ముగిసింది. TTL చాలా పెద్దదైతే అమ్మకాలు **35% పడిపోయాయి**, ఎందుకంటే చెల్లించనివాళ్ళ holds సీట్లని బంధించాయి. కాబట్టి TTL ని నేను ఎంచుకోను - నేను చెల్లింపు p99 ని కొలిచి దానికి సమానంగా పెడతాను."

ఇది ఒక పరిష్కారం చెప్పడం కాదు - ఆ పరిష్కారానికి ఒక ధర ఉందని మీరు చూశారని చూపుతుంది.

14. నీటితో చెప్పాల్సిన English script

Scope గురించి:

"I'd start by pointing out that this is the smallest dataset I'd ever be asked to design for – about eight gigabytes, averaging twelve writes a second. A single database handles that. What makes it hard is that the peak is fifty thousand times the average and all of it lands on the same few thousand rows. Sharding doesn't help, because the seats for one show have to live together for anyone to decide who owns them. Caching doesn't help, because a stale answer here means selling the same seat twice."

Overselling గురించి:

"The natural implementation reads the seat, checks it's free, then writes. I ran that with three thousand users against a thousand seats: it sold sixty thousand tickets for twenty thousand seats – forty thousand double-sales. Every individual operation was valid; the bug lives in the gap between them. So the check and the write have to be one atomic step – a conditional update on a version column, or a row lock. Both are correct. What I'd flag is granularity: locking the whole show is also correct and it was three hundred thirty-seven times slower in my measurement, because it serialises the entire sale."

Holds గురించి:

"Then holds, because payment takes time. And here's where a correct system still fails completely. I ran ten times more demand than seats, with sixty percent of users abandoning checkout, and no expiry on holds: sixty percent of seats never sold. No overselling, no lost updates, nothing 'wrong' – just an empty venue and a sold-out website. The fix is a TTL, but a TTL opens a race: your hold expires, someone else takes the seat, and *then* your payment succeeds. Without re-checking ownership at confirm time, I measured two thousand double-sales again. And the TTL value itself has damage on both sides – too short and seventy-nine percent of payments fail after taking money; too long and sales drop thirty-five percent. So I wouldn't pick a TTL; I'd measure the p99 payment time and set it to that."

The surge గురించి:

"Finally the on-sale. Two hundred thousand people, ten thousand seats. With immediate retries the system processed two point nine million requests, ninety-nine percent of them wasted. Exponential backoff cut that by a third and it was still ninety-nine percent waste. A virtual waiting room cut it fifteen-fold. But the number I'd actually lead with is the last column: the sell-out time was identical in all three – fourteen hundred ticks, within zero point two percent. The retry storm bought nothing, because the sale is paced by hold expiry, not by request throughput. So the question isn't how to serve more requests. It's why those requests exist at all."

15. Follow-ups – పక్కపక్క సీట్లు, bots, డబ్బు వాపసు

"నలుగురు కలిసి పక్కపక్కన కూర్చోవాలంటే?"

ఇప్పుడు ఇది ఒక సీటు కాదు, ఒక **పరిధి** – మరియు మీరు 4 సీట్లని **అన్నీ కలిసి** పట్టుకోవాలి, లేదా ఏదీ కాదు. నాలుగింటిని వరుసగా పట్టుకుంటే **deadlock** వస్తుంది (A సీట్లు 5,6 పట్టుకున్నాడు, B సీట్లు 7,6 పట్టుకున్నాడు, ఇద్దరూ వేచి ఉంటారు). రెండు ప్రామాణిక పరిష్కారాలు: సీట్లని **ఎప్పుడూ పెరుగుతున్న క్రమంలో** పట్టుకోవడం (deadlock అసాధ్యం అవుతుంది), లేదా **మొత్తం block కి ఒకే అణు ఆపరేషన్**. రెండేది సరళమైనది, మరియు అది §5 యొక్క lock ని కొంచెం వెడల్పు చేస్తుంది – అదే మనం తప్పించాలనుకున్నది. **ఇది నేను ఈ doc lo కొలవలేదు.**

"Bots మొత్తం inventory తీసుకుంటే?"

ఒక్కే ఖాతాకి పరిమితి, ఒక్కే payment పద్ధతికి పరిమితి, మరియు §11 యొక్క వేచి ఉండే గదిలో ప్రవేశానికి ముందే గుర్తింపు. ముఖ్యమైన architectural పరిణామం: ఈ తనిఖీలు **వేచి ఉండే గది ప్రవేశద్వారం** దగ్గర ఉండాలి, booking దగ్గర కాదు – లేకపోతే bot traffic ఇప్పటికే మీ ఖరీదైన మార్గంలోకి చేరిపోయింది.

"డబ్బు తీసుకున్నాక సీటు ఇవ్వలేకపోతే?"

§8 ప్రకారం సరైన TTL తో ఇది 0%. కానీ 0% అనేది ఒక నమూనా – నిజంగా payment gateway timeout అవుతుంది, webhook ఆలస్యమవుతుంది. కాబట్టి **స్వయంచాలక refund** ఒక design అవసరం, ఐచ్ఛికం కాదు. మరియు అది **idempotent** కావాలి (HLD Deep 09). ఉత్తమమైన పద్ధతి: చెల్లింపుని **రెండు దశలుగా** – ముందు **authorize** (డబ్బు పట్టుకోవడం, తీసుకోవడం కాదు), సీటు ఖాయమయ్యాక **capture**. అప్పుడు విఫలమైతే కేవలం authorization వదిలేస్తే చాలు.

"సీట్ల map ని ఎలా చూపిస్తారు?"

ఇది ఒక cache ప్రశ్న, మరియు ఇక్కడ **పాత data సరైనదే**. "ఈ సీటు ఖాళీగా ఉంది" అనేది ఒక సూచన, ఒక హామీ కాదు – నిజమైన నిర్ణయం §5 యొక్క అణు ఆపరేషన్ దగ్గర జరుగుతుంది. కాబట్టి map ని 1-2 సెకన్ల cache తో CDN నుంచి ఇవ్వొచ్చు, మరియు అది మీ database మీద 99% load తగ్గిస్తుంది. **"చదవడం పాతది కావచ్చు, రాయడం కాదు"** – ఇదే ఈ problem lo cache యొక్క సరైన స్థానం.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **పక్కపక్క సీట్ల కేటాయింపు** (పైన), **distributed lock** (నా model ఒక్క process – నిజమైన systems lo ఇది Redis లేదా database మీద ఆధారపడుతుంది, మరియు ఆ lock యొక్క విఫలత ఒక పూర్తి doc), మరియు **payment gateway యొక్క నిజమైన latency పంపిణీ** (నా 20-400 ticks ఒక ఏకరీతి నమూనా, నిజమైనది కాదు – మరియు 98 యొక్క optimum ఆ పంపిణీ మీద నేరుగా ఆధారపడుతుంది).

16. ఏమి నేర్చుకున్నాం

1. చిన్న data పెద్ద problem కావచ్చు. 8 GB, సెకనుకి 12 writes – మరియు ఈ series lo అత్యంత కష్టమైన concurrency.
2. దోషం ఆపరేషన్లలో కాదు, వాటి మధ్య ఖాళీలో ఉంటుంది. ప్రతి UPDATE చెల్లుబాటు అయ్యేదే. ఫలితం 40,964 రెండుసార్లు అమ్మకాలు.
3. సరైనది సరిపోదు – lock ఎంత ఇరుకో అది కూడా ముఖ్యం. Show-lock సరైనది, మరియు 337 రెట్లు నెమ్మది.
4. "ఏదీ తప్పు జరగలేదు" మళ్ళీ సరిపోలేదు. Oversell లేదు, corruption లేదు – మరియు 60% సీట్లు ఖాళీగా ఉండిపోయాయి.
5. ఒక పరిష్కారం ఒక కొత్త race ని తెరవగలదు. TTL inventory ని కాపాడింది, మరియు 94 యొక్క విరుపుని తిరిగి తెచ్చింది – 2,068 రెండుసార్లు.
6. ప్రతి timeout కి రెండు వైపులా నష్టం ఉంది. చిన్నది → 78.8% విఫల చెల్లింపులు. పెద్దది → 35% తక్కువ అమ్మకాలు. దాన్ని ఎంచుకోకండి – p99 ని కొలవండి.
7. ఎక్కువ requests ఒక బాహ్య పరిమితిని మార్చవు. 29 లక్షల requests, 99.1% వృథా, మరియు అమ్మకం ఒక్క tick కూడా వేగంగా ముగియలేదు.
8. ఒక mutation బతికితే, మే model ఆ సమస్యని ఊహించి తొలగించేసిందేమో చూడండి. నా `hold()` అణువు, కాబట్టి version తనిఖీ అనవసరం – మరియు అది నిజ ప్రపంచంలో 94 యొక్క 40,964 ని ఆపేది.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ problem lo ప్రతి విరుపూ ఒకే ఆకారంలో ఉంది: రెండు అడుగుల మధ్య ఒక ఖాళీ.

94 – చదవడం మరియు రాయడం మధ్య ఖాళీ → రెండుసార్లు అమ్మకం.

97 – hold మరియు చెల్లింపు మధ్య ఖాళీ → ఖాళీ సభ.

98 – గడువు ముగియడం మరియు చెల్లింపు పూర్తవడం మధ్య ఖాళీ → డబ్బు తీసుకుని సీటు లేదు.

ప్రతిసారీ పరిష్కారం ఒకటే ఆకారం: ఆ రెండు అడుగులని ఒక్క అణు నిర్ణయంగా చేయడం, లేదా ఆ ఖాళీకి ఒక స్పష్టమైన గడువు పెట్టడం.

Concurrency lo "అపై" అనే పదం ఎప్పుడూ ఒక ప్రమాదం.